Assignment No. 02

AI4003 - Applied Machine Learning

Fall 2026

Deadline: 11 October 2026

Detecting Faults on a Three-Phase Transmission Line
(logistic regression by hand and support vector machines)

Targeted Course Learning Outcome

CLO4: Design solutions using machine learning models with supervised or unsupervised techniques to address engineering challenges. **(C6)**.

Submission Instructions:

- This is a group assignment.
- Each group should be exactly 2 members.
- **Plagiarism is strictly prohibited. Plagiarism among groups would be checked with tools.**
- **Please do not use any AI tool for direct copy / paste and do not copy from others.**
- **AI plagiarism using tools such as ChatGPT, Claude, Gemini, Copilot, Cursor, etc. will be marked straight zero.**
- **Submission carries no marks. Marks will be awarded based on viva.**
- Write a Medium Blog and a LinkedIn Post and submit their links.
- Develop the front end, and push all code to GitHub with commits from both team members. Submit Repo Link.

Read the whole manual before you begin.

Contents

1 Overview **3**
- 1.1 The problem . . . 3
- 1.2 Dataset . . . 3
- 1.3 Learning outcomes . . . 3

2 Instructions **4**
- 2.1 Task 1 – Load and prepare the data . . . 4
- 2.2 Task 2 – Exploratory plots . . . 4
- 2.3 Task 3 – Logistic regression by hand . . . 5
- 2.4 Task 4 – Support vector machines . . . 5
- 2.5 Task 5 – Test once . . . 6
- 2.6 Task 6 – Analysis . . . 6
- 2.7 Task 7 – Front end and communication . . . 6
- 2.8 Required repository layout . . . 7

3 Evaluation **7**
- 3.1 Metrics . . . 7
- 3.2 Correctness checks . . . 7

4 Results **8**
- 4.1 Table 1 – Data preparation . . . 8
- 4.2 Table 2 – Logistic regression . . . 8
- 4.3 Table 3 – SVM validation results . . . 8
- 4.4 Table 4 – Test results . . . 8
- 4.5 Table 5 – Learned θ (Set Q) . . . 8
- 4.6 Figures . . . 9

Appendix: Data Loader **10**

1. Overview

1.1 The problem

When a fault occurs on a transmission line (a phase touching ground, or two phases touching each other) the line currents jump and the voltages sag. A protective relay must see this within milliseconds and trip the breaker. In this assignment you will build a classifier that looks at one instant of the three line currents and three line voltages and decides:

Input	I_a, I_b, I_c (A) and V_a, V_b, V_c (per unit) at one instant
Output	$y = 1$ (fault) or $y = 0$ (no fault)

and answer one practical question:

The question

Can a straight-line decision boundary detect a fault from raw instantaneous measurements? If not, what is the simplest fix: changing the features, or changing the classifier?

You will implement **logistic regression** yourself with NumPy and train **support vector machines** (linear and RBF kernel) with scikit-learn.

1.2 Dataset

Electrical Fault Detection and Classification (Kaggle): a three-phase power system simulated in MATLAB/Simulink. Each row is one instant of the line currents and voltages, labelled fault or no fault.

Download	https://www.kaggle.com/datasets/esathyaprakash/electrical-fault-detection-and-classification
File used	`detect_dataset.csv` (about 12,000 rows)

Column	Meaning
`Output (S)`	label: 1 = fault, 0 = no fault
`Ia`, `Ib`, `Ic`	instantaneous line currents of phases A, B, C
`Va`, `Vb`, `Vc`	instantaneous line voltages of phases A, B, C

The same Kaggle page also has `classData.csv`; it is **not** used in this assignment. A loader that reads `detect_dataset.csv` is given in the Appendix; everything after loading is your work.

1.3 Learning outcomes

1. Clean a real CSV file and split it correctly into train, validation and test sets.
2. Implement logistic regression from the sigmoid and the log-likelihood.
3. Understand why a linear classifier fails on data that is not linearly separable.
4. Fix it in two ways: new features, or a kernel SVM.
5. Choose hyper-parameters (α, C, γ) with a validation set.
6. Evaluate a classifier with accuracy and a confusion matrix.

2. Instructions

Complete the tasks in order. Every task lists exactly what must appear in your repository.

Allowed libraries

`numpy`, `pandas`, `matplotlib`, `joblib`, a front-end framework of your choice, and from scikit-learn **only** `sklearn.svm.SVC`.

Not allowed anywhere: `LogisticRegression`, `train_test_split`, `StandardScaler`, anything from `sklearn.metrics`, or any other library that splits data, scales features, fits models or computes metrics for you.

2.1 Task 1 – Load and prepare the data

1.1 Download `detect_dataset.csv` from Kaggle into `data/` and load it with the loader in the Appendix. Print the shape, the column names and the first five rows.

1.2 Inspect the columns yourself. Remove any column that carries no information, and rename the label column to `fault`. Check that all six measurement columns are numeric and report the number of missing values per column.

1.3 Report how many rows are `fault = 1` and how many are `fault = 0`.

1.4 **Split** the rows with exactly this code, so every group has the same split:

```
rng = np.random.default_rng(42)
idx = rng.permutation(len(df))
n_train, n_val = int(0.6 * len(df)), int(0.2 * len(df))
train_idx = idx[:n_train]
val_idx   = idx[n_train:n_train + n_val]
test_idx  = idx[n_train + n_val:]
```

Report the number of rows and the fraction of faults in each split.

1.5 **Standardise** every feature: subtract the training mean and divide by the training standard deviation. Use the *training* statistics for all three splits.

2.2 Task 2 – Exploratory plots

Produce the following three plots from the *raw* (unstandardised) data. Under each, write two or three sentences explaining what you see.

2.1 I_a, I_b, I_c against row number for the first 2000 rows, with the background shaded wherever `fault = 1`. (The rows are consecutive time samples.)

2.2 Scatter plot of I_a against I_b, coloured by `fault`.

2.3 Histogram of I_a for fault rows and for no-fault rows on the same axes.

After these plots, answer in one paragraph: *could a single straight line in the (I_a, I_b) plane separate the two classes? Why, in terms of what a fault does to the current?*

2.3 Task 3 – Logistic regression by hand

Notation (as in the lecture). $x^{(i)}$ is the feature vector of row i with $x_0 = 1$, $y^{(i)} \in \{0, 1\}$, and n is the number of training rows.

$$g(z) = \frac{1}{1 + e^{-z}}, \qquad h_\theta(x) = g(\theta^\top x),$$

$$\ell(\theta) = \sum_{i=1}^{n} \Big[y^{(i)} \log h_\theta(x^{(i)}) + \big(1 - y^{(i)}\big) \log\big(1 - h_\theta(x^{(i)})\big) \Big], \qquad J(\theta) = -\frac{1}{n}\, \ell(\theta).$$

Two feature sets:

Set R (raw, 6 features)	$I_a, I_b, I_c, V_a, V_b, V_c$
Set Q (raw + squares, 12 features)	Set R and $I_a^2, I_b^2, I_c^2, V_a^2, V_b^2, V_c^2$

For Set Q, compute the squares from the *raw* values first, then standardise all 12 columns using training statistics.

3.1 Write `sigmoid(z)`, `hypothesis(X, theta)` and `cost(X, y, theta)`. Inside `cost`, clip h to $[10^{-12}, 1 - 10^{-12}]$ before taking the log.

3.2 **Batch gradient descent.** Write `fit_logistic(X, y, alpha, n_iters)` that starts from $\theta = \vec{0}$ and repeats, for every j,

$$\theta_j := \theta_j + \frac{\alpha}{n} \sum_{i=1}^{n} \big(y^{(i)} - h_\theta(x^{(i)})\big)\, x_j^{(i)}.$$

Record $J(\theta)$ after every iteration. (Note that this is the same form as the LMS rule from Lecture 1, with a different h_θ.)

3.3 Write `predict(X, theta)` that returns 1 if $h_\theta(x) \geq 0.5$ and 0 otherwise.

3.4 **Choose α.** For each feature set, run 2000 iterations with $\alpha \in \{0.01,\ 0.1,\ 1,\ 10\}$ on the training rows. Plot $J(\theta)$ against iteration for all four values on one figure per feature set. Pick the **largest** α for which $J(\theta)$ decreases at every iteration.

3.5 Train the final model for each feature set with its chosen α and 2000 iterations. Report training and validation accuracy, and print θ.

2.4 Task 4 – Support vector machines

Use `sklearn.svm.SVC` on **Set R only** (the 6 raw standardised features). Fit on the training rows, measure accuracy on the validation rows.

4.1 **Linear SVM:** `SVC(kernel="linear", C=C)` for $C \in \{0.1,\ 1,\ 10,\ 100\}$.

4.2 **RBF SVM:** `SVC(kernel="rbf", C=C, gamma=gamma)` for $C \in \{0.1,\ 1,\ 10,\ 100\}$ and $\gamma \in \{0.1,\ 1\}$ (8 models).

4.3 Put the results in a table: C, γ, validation accuracy, and the number of support vectors (`model.n_support_.sum()`).

4.4 For each kernel, keep the setting with the highest validation accuracy. Save both models with `joblib.dump`.

2.5 Task 5 – Test once

5.1 Write the functions `accuracy(y_true, y_pred)` and `confusion_matrix(y_true, y_pred)` yourself (definitions in Section 3).

5.2 Evaluate the four final models on the **test** rows, once: logistic regression on Set R, logistic regression on Set Q, the best linear SVM, and the best RBF SVM.

5.3 For each, report accuracy, the confusion matrix, and the number of **missed faults** (actual fault, predicted no fault).

2.6 Task 6 – Analysis

Answer in the blog and in `results/analysis.md`:

6.1 Logistic regression on Set R and the linear SVM should score almost the same. Why? What does that tell you about the data?

6.2 Look at your Set Q weights. What sign do the weights on I^2 terms have, and what sign do the weights on V^2 terms have? Explain both signs using what happens to current and voltage during a fault.

6.3 Compare the number of support vectors of your best linear SVM with your best RBF SVM. What does a very large number of support vectors mean?

6.4 From your validation table, what happens to accuracy and to the number of support vectors as C increases? Explain using what C controls.

6.5 The rows are consecutive time samples, and the split was random, so neighbouring samples (which look almost identical) land in both training and test sets. Do you think your test accuracy is optimistic or pessimistic for a real relay? Why?

6.6 If you were designing the relay, which model would you deploy? Base your answer on missed faults and false alarms, not only accuracy.

2.7 Task 7 – Front end and communication

7.1 A minimal web interface (Streamlit, Gradio, Flask, ...) where the user enters the six values $I_a, I_b, I_c, V_a, V_b, V_c$ and sees the prediction of *your* saved Set-Q logistic regression and *your* saved RBF SVM, side by side. Screenshot in the README.

7.2 **Medium blog** (800–1500 words): the problem, the plots from Task 2, the results table, and your answers to Task 6. At least two figures.

7.3 **LinkedIn post** linking the blog and repository.

7.4 **GitHub repository**: public; README with run instructions; meaningful commits from *both* members.

2.8 Required repository layout

`data/`	`detect_dataset.csv`
`src/load_data.py`	the given loader
`src/prepare.py`	Task 1 (cleaning, split, standardisation)
`src/eda.py`	Task 2
`src/logistic.py`	`sigmoid`, `hypothesis`, `cost`, `fit_logistic`, `predict`
`src/metrics.py`	`accuracy`, `confusion_matrix`
`src/train_logistic.py`	Task 3
`src/train_svm.py`	Task 4
`src/test_models.py`	Task 5
`results/`	tables, figures, `analysis.md`, saved models and θ
`app/`	Task 7 front end
`README.md`	how to run everything

3. Evaluation

3.1 Metrics

Training: the cost $J(\theta)$ for logistic regression, recorded during gradient descent.

Validation and test: with m rows, true labels $y^{(i)}$ and predictions $\hat{y}^{(i)}$,

$$\text{Accuracy} = \frac{1}{m}\sum_{i=1}^{m} \mathbf{1}\left[\hat{y}^{(i)} = y^{(i)}\right].$$

The **confusion matrix** counts the four possible outcomes:

	Predicted 0 (no fault)	**Predicted 1 (fault)**
Actual 0 (no fault)	correct	false alarm
Actual 1 (fault)	**missed fault**	correct

A missed fault is the dangerous error for a protection system: the breaker does not trip.

Expected range

With the split above, a correct implementation gives roughly: logistic regression on Set R and the linear SVM around 70–75% test accuracy, with hundreds of missed faults; logistic regression on Set Q above 95%; the best RBF SVM above 98%. If Set R logistic regression scores above 90%, check that the label column was not left in the features. If Set Q scores about the same as Set R, check that the squared columns were actually added to the feature matrix.

3.2 Correctness checks

Before submitting, confirm:

- The two empty columns and the label are not among the features.
- Standardisation statistics come from training rows only.
- $J(\theta)$ starts at $\log 2 \approx 0.693$ when $\theta = \vec{0}$.
- $J(\theta)$ decreases at every iteration with your chosen α.

- Your confusion matrix entries add up to the number of test rows.
- The test rows are never used for training or for choosing α, C or γ.

4. Results

Present the following in `results/` and in the blog, using these layouts.

4.1 Table 1 – Data preparation

Raw rows / raw columns
Columns removed (names)
Missing values after cleaning
Fault rows / no-fault rows
Train / validation / test rows
Fault fraction in train / validation / test

4.2 Table 2 – Logistic regression

Feature set	**Chosen α**	**Final $J(\theta)$**	**Validation accuracy**
Set R			
Set Q			

4.3 Table 3 – SVM validation results

Kernel	C	γ	**Validation accuracy**	**Support vectors**
linear	0.1	–		
linear	1	–		
linear	10	–		
linear	100	–		
rbf	0.1	0.1		
rbf	...	...		
rbf	100	1		

4.4 Table 4 – Test results

Model	**Accuracy**	**TN**	**False alarms**	**Missed faults**	**TP**
Logistic regression, Set R					
Logistic regression, Set Q					
Linear SVM (C=__)					
RBF SVM (C=__, γ=__)					

4.5 Table 5 – Learned θ (Set Q)

One row per weight, in this order:

$$\theta_0 \text{ (intercept)},\ I_a,\ I_b,\ I_c,\ V_a,\ V_b,\ V_c,\ I_a^2,\ I_b^2,\ I_c^2,\ V_a^2,\ V_b^2,\ V_c^2.$$

4.6 Figures

1. The three exploratory plots from Task 2.
2. $J(\theta)$ against iteration for four values of α, Set R and Set Q (Task 3.4).
3. Validation accuracy against C for the linear SVM and for the RBF SVM (both γ values), on one figure with C on a log scale.
4. Front-end screenshot.

Appendix: Data Loader

This is the only code provided. It reads the CSV file as it is; it does **not** rename, drop, convert or clean anything.

A.1 Environment

```
pip install numpy pandas matplotlib scikit-learn joblib
```

A.2 Getting the data

1. Create a free Kaggle account and open the dataset page (link in Section 1).
2. Click **Download**, unzip, and place `detect_dataset.csv` in `data/`.

A.3 `src/load_data.py`

```python
from pathlib import Path
import pandas as pd

FILE = "detect_dataset.csv"

def load_raw(data_dir="data"):
    """Return the raw fault-detection DataFrame exactly as stored in
       the CSV.

    Nothing is renamed, dropped, converted or cleaned.
    """
    path = Path(data_dir) / FILE
    if not path.exists():
        raise FileNotFoundError(f"Missing {path}. Download the
           dataset first.")
    return pd.read_csv(path)

if __name__ == "__main__":
    df = load_raw()
    print(f"{df.shape[0]} rows, {df.shape[1]} columns")
    print(df.columns.tolist(), end="\n\n")
    print(df.head(5).to_string())
```

Run `python src/load_data.py` once to confirm the file loads. Look carefully at the printed column names before you begin Task 1.

Good luck. A relay that misses a fault is worse than one that trips too often.